Completing the Cohomological Extension Package:

Section Cocycles and Splitting Criteria for Mathlib

Nova Spivack

2026

Abstract

Mathlib's `GroupExtension/Defs.lean` documents two explicit TODO items: a bijection between N-conjugacy classes of splittings and H^1, and a bijection between equivalence classes of group extensions and H^2, both for abelian kernel N. Neither is currently formalized. This technical note documents a proposed Mathlib contribution that constitutes **Phase 1** of completing these TODOs: the *section cocycle* associated to a set-theoretic section of a group extension, the *splitting criterion* (`splits_iff_trivial_cocycle`), and supporting infrastructure including the section difference function, the conjugation action via sections, and the multiplicative 2-cocycle identity. All theorems are fully verified by the Lean type checker with zero `sorry`. We describe the proposed API surface, design decisions (namespace placement, `@[to_additive]` strategy, `@[simp]` candidates), the relationship to existing Mathlib infrastructure (`GroupExtension.Defs`, `GroupExtension.Basic`, `GroupCohomology.LowDegree`), and a roadmap for Phase 2 (the H^1 and H^2 bridges). The contribution builds against Mathlib v4.29.0-rc6.

Keywords: Mathlib, group extensions, section cocycle, splitting criterion, group cohomology, Lean 4

MSC 2020: 20J06, 18G50, 68V15

Code/Data availability: https://github.com/novaspivack/infinity-compression — see `GroupExtension/` modules

Contents

1 Contribution Summary **3**
- 1.1 The Mathlib gap . . . 3
- 1.2 What we contribute . . . 3
- 1.3 Relationship to the TODOs . . . 3

2 Design Decisions **4**
- 2.1 Namespace placement . . . 4
- 2.2 Reuse of existing membership lemmas . . . 4
- 2.3 Why `noncomputable` . . . 4
- 2.4 `@[to_additive]` strategy . . . 4
- 2.5 `@[simp]` candidates . . . 5
- 2.6 Universe polymorphism . . . 5

3 API Surface **5**
- 3.1 Lean code: core definitions . . . 7
- 3.2 Lean code: the splitting criterion . . . 8

4 Relationship to Existing Infrastructure **8**
4.1 Dependency on `GroupExtension.Defs` . . . 8
4.2 Dependency on `GroupExtension.Basic` . . . 9
4.3 Relationship to `GroupCohomology.LowDegree` . . . 9
4.4 Composition diagram . . . 10

5 Dependency Structure and Proof Architecture **10**
5.1 Proof strategy overview . . . 10

6 Toward H^1 and H^2: Phase 2 Roadmap **11**
6.1 Phase 2a: The H^1 bridge . . . 11
6.2 Phase 2b: The H^2 bridge . . . 11
6.3 Estimated effort . . . 12

7 Testing and Compatibility **12**
7.1 Build status . . . 12
7.2 Proof status . . . 12
7.3 Module structure . . . 13
7.4 Interaction with existing `simp` lemmas . . . 13
7.5 AI use disclosure . . . 13

1 Contribution Summary

1.1 The Mathlib gap

Mathlib's `Mathlib.GroupTheory.GroupExtension.Defs` (lines 43–52 as of v4.29.0-rc6) contains the following documented TODO:

> *If N is abelian,*
>
> - *there is a bijection between N-conjugacy classes of* `(SemidirectProduct.toGroupExtension` φ`).Splitting` *and* `groupCohomology.H1`
> - *there is a bijection between equivalence classes of group extensions and* `groupCohomology.H2`

These are the two classical cohomological classification theorems for group extensions. The first identifies the "freedom in choosing a splitting" with the first cohomology group; the second identifies the "space of all extensions with a given kernel action" with the second cohomology group. Neither is formalized in Mathlib.

1.2 What we contribute

Our contribution is **Phase 1**: the foundational layer that both classification theorems require. Concretely, we add:

1. **The section cocycle** (`Section.cocycle`): Given a group extension $1 \to N \to E \to G \to 1$ and a set-theoretic section $\sigma : G \to E$, the function $c_\sigma(g_1, g_2) = \mathrm{inl}^{-1}(\sigma(g_1) \cdot \sigma(g_2) \cdot \sigma(g_1 g_2)^{-1})$ is the N-valued 2-cocycle measuring the failure of σ to be a homomorphism.
2. **The splitting criterion** (`splits_iff_trivial_cocycle`): The extension splits (admits a homomorphic section) if and only if some set-theoretic section has trivial cocycle.
3. **The section difference** (`Section.diff`): The N-valued function measuring the pointwise difference between two sections.
4. **The conjugation action via sections** (`Section.conjAct`): The action of G on N obtained by conjugating through a section, composing Mathlib's existing `GroupExtension.conjAct` with the section.
5. **The 2-cocycle identity** (`Section.cocycle_mul_eq`): The multiplicative 2-cocycle condition

   $$c(g_1, g_2) \cdot c(g_1 g_2, g_3) = \varphi_\sigma(g_1)(c(g_2, g_3)) \cdot c(g_1, g_2 g_3)$$

   connecting our cocycle to Mathlib's $\texttt{IsMulCocycle}_2$.

1.3 Relationship to the TODOs

The H^2 bridge requires: (a) defining the section cocycle, (b) proving it satisfies the 2-cocycle identity, (c) showing the cohomology class is section-independent (i.e., that changing the section changes the cocycle by a coboundary), and (d) constructing the bijection with equivalence classes of extensions. Our contribution covers (a) and proves (b), along with the infrastructure for (c) (the section difference function). The cocycle change formula and bijection construction remain for Phase 2. The H^1 bridge requires: (a) the conjugation action via sections, (b) the section difference function, and (c) constructing the bijection with conjugacy classes of splittings. Our contribution covers (a)–(b). In both cases, we provide the algebraic infrastructure; the coboundary formula and bijection constructions remain for Phase 2.

2 Design Decisions

2.1 Namespace placement

All new definitions and theorems are placed under `GroupExtension.Section` or `GroupExtension`, following Mathlib's existing namespace structure:

Pattern	Example
Section-specific definitions	`GroupExtension.Section.cocycle`
Section-specific theorems	`GroupExtension.Section.cocycle_spec`
Extension-level theorems	`GroupExtension.splits_iff_trivial_cocycle`

This mirrors the existing pattern where `GroupExtension.Section` holds section-level API and `GroupExtension` holds extension-level results (e.g., `GroupExtension.conjAct` in `Defs.lean`, `GroupExtension.surjInvRightHom` in `Basic.lean`).

2.2 Reuse of existing membership lemmas

Mathlib's `Basic.lean` already provides two membership lemmas that are central to our definitions:

- `Section.mul_mul_mul_inv_mem_range_inl`: establishes that $\sigma(g_1)\cdot\sigma(g_2)\cdot\sigma(g_1g_2)^{-1} \in \mathrm{range}(\texttt{inl})$. This is the membership fact underlying `Section.cocycle`.
- `Section.mul_inv_mem_range_inl`: establishes that $\sigma(g) \cdot \sigma'(g)^{-1} \in \mathrm{range}(\texttt{inl})$. This is the membership fact underlying `Section.diff`.

Our current development re-derives these facts (as `section_defect_mem_range` and `section_diff_mem_range`) because the code lives in a separate namespace. The Mathlib PR would use the existing lemmas directly, applying `MonoidHom.mem_range.mp` to extract the `Exists` witness for `Exists.choose`.

2.3 Why `noncomputable`

The cocycle and section difference are defined via `Exists.choose` on a membership proof in `S.inl.range`. Since `inl` is injective, the chosen preimage is unique, but the extraction is still noncomputable because `Exists.choose` is noncomputable in Lean 4. This is standard Mathlib practice for definitions that extract elements from existential proofs. The alternative—carrying the membership proof as data—would complicate the API without benefit, since all downstream theorems work through the specification lemma (`cocycle_spec`) rather than computing the cocycle directly.

```
noncomputable def cocycle (g₁ g₂ : G) : N :=
  (mul_mul_mul_inv_mem_range_inl σg₁ g₂).choose
```

2.4 `@[to_additive]` strategy

Mathlib's `GroupExtension` is defined multiplicatively. The `@[to_additive]` attribute should be applied to all new definitions and theorems to generate the additive counterparts automatically. The naming convention follows Mathlib's standard: the multiplicative name uses `mul` and the additive name uses `add`.

Multiplicative	Additive (generated)
`GroupExtension.Section.cocycle`	`AddGroupExtension.Section.cocycle`
`splits_iff_trivial_cocycle`	`splits_iff_trivial_cocycle`
`Section.cocycle_mul_eq`	`Section.cocycle_add_eq`

Implementation note. Our current development does not carry `@[to_additive]` annotations. Mathlib v4.29.0-rc6 does define `AddGroupExtension` and its `Section`/`Splitting` structures, and many lemmas in `Defs.lean` and `Basic.lean` already carry `@[to_additive]`. However, `conjAct` and `inl_conjAct_comm` (which our development depends on) do *not* carry `@[to_additive]`, so the annotations cannot be applied uniformly to our full API without first adding additive counterparts for those upstream definitions. The PR should either (a) add the missing additive `conjAct` infrastructure upstream, or (b) apply `@[to_additive]` only to the definitions that do not depend on `conjAct` (`cocycle`, `diff`, `splits_iff_trivial_cocycle`) and defer the rest.

2.5 `@[simp]` candidates

The following lemmas are candidates for the `@[simp]` attribute:

1. `Section.cocycle_spec`: The defining equation $\texttt{S.inl}(c_\sigma(g_1, g_2)) = \sigma(g_1) \cdot \sigma(g_2) \cdot \sigma(g_1 g_2)^{-1}$ is the primary rewriting rule for the cocycle. It rewrites an `S.inl` application into section values, which is the typical simplification direction.
2. `Section.diff_spec`: Similarly, $\texttt{S.inl}(\mathrm{diff}_{\sigma,\sigma'}(g)) = \sigma(g) \cdot \sigma'(g)^{-1}$ rewrites `S.inl` of a diff into section values.
3. `sectionCocycle_of_splitting`: The cocycle of a splitting's section is trivial: $c_s(g_1, g_2) = 1$. This is a strong simplification that eliminates cocycles when a splitting is known.

Non-candidates:

- `Section.mul_eq_cocycle_mul`: This rewrites $\sigma(g_1) \cdot \sigma(g_2)$ into $\texttt{S.inl}(c(g_1, g_2)) \cdot \sigma(g_1 g_2)$, which introduces `S.inl` terms. As a `simp` lemma it would conflict with `cocycle_spec` (which eliminates `S.inl` terms), creating a loop. It should be a `rw` target only.
- `splits_iff_trivial_cocycle`: This is a biconditional characterization theorem, not a simplification rule. It should be used via `rw` or `iff` reasoning.

2.6 Universe polymorphism

Our development uses a single explicit universe variable `u` for N, E, and G. Mathlib's `GroupExtension` uses `Type*` (auto-bound implicit universes), which allows N, E, and G to live in different universes. For the PR, the definitions should be updated to use `Type*` to match Mathlib convention. In practice, all our theorems work with a single universe, so this is a cosmetic change.

3 API Surface

Table 1 lists all proposed new definitions and Table 2 lists all proposed new theorems, with their full Lean 4 types.

Table 1: Proposed new definitions.

Name	Type
`GroupExtension.Section.cocycle`	`S.Section → G → G → N` The N-valued 2-cocycle of a set-theoretic section. Defined as the `inl`-preimage of $\sigma(g_1) \cdot \sigma(g_2) \cdot \sigma(g_1 g_2)^{-1}$. Noncomputable (via `Exists.choose`).
`GroupExtension.Section.diff`	`S.Section → S.Section → G → N` The N-valued difference between two sections. Defined as the `inl`-preimage of $\sigma(g) \cdot \sigma'(g)^{-1}$. Noncomputable.
`GroupExtension.Section.conjAct`	`S.Section → G → MulAut N` The conjugation action of G on N via a section: $\varphi_\sigma(g) =$ `S.conjAct`$(\sigma(g))$. Composes `GroupExtension.conjAct : E →* MulAut N` (from `Defs.lean`) with the section. Not a group homomorphism for a general set-theoretic section (cf. `Splitting.conjAct` in `Basic.lean` which is).

Table 2: Proposed new theorems.

Name	Statement
`Section.cocycle_spec`	`S.inl (cocycle` σ g_1 g_2`) =` σ`.toFun` g_1 `*` σ`.toFun` g_2 `* (`σ`.toFun (`g_1 `*` g_2`))`$^{-1}$ Defining equation of the cocycle. Candidate for `@[simp]`.
`Section.mul_eq_cocycle_mul`	σ`.toFun` g_1 `*` σ`.toFun` g_2 `= S.inl (cocycle` σ g_1 g_2`) *` σ`.toFun (`g_1 `*` g_2`)` Decomposition of section products. Rearrangement of `cocycle_spec`.
`splits_iff_trivial_cocycle`	`Nonempty S.Splitting` $\leftrightarrow$ $\exists$ σ `: S.Section,` $\forall$ g_1 g_2`, cocycle` σ g_1 g_2 `= 1` The central splitting criterion.
`Section.diff_spec`	`S.inl (diff` σ σ' `g) =` σ`.toFun g *` $(\sigma'$`.toFun g)`$^{-1}$ Defining equation of the section difference. Candidate for `@[simp]`.
`Section.conjAct_spec`	`S.inl (conjAct` σ `g n) =` σ`.toFun g * S.inl n *` $(\sigma$`.toFun g)`$^{-1}$ Defining equation of the conjugation action via sections. Follows from Mathlib's `inl_conjAct_comm`.
`Section.cocycle_mul_eq`	`cocycle` σ g_1 g_2 `* cocycle` σ `(`g_1 `*` g_2`)` g_3 `= conjAct` σ g_1 `(cocycle` σ g_2 g_3`) * cocycle` σ g_1 `(`g_2 `*` g_3`)`

Name	Statement
	The multiplicative 2-cocycle identity with respect to the conjugation action. This is the equation required by `IsMulCocycle₂` (after uncurrying and extracting the `MulDistribMulAction`).
`sectionCocycle_of_splitting`	$\forall$ g_1 g_2, `cocycle s.toSection` g_1 g_2 `= 1`
	The cocycle of a splitting's section is trivial. Candidate for `@[simp]`.
`Section.conjAct_independent`	`[CommGroup N]` $\to$ `conjAct` σ `g n = conjAct` σ' `g n`
	For abelian N, the conjugation action is independent of section choice.
`section_triple_product`	σ g_1 `*` σ g_2 `*` σ g_3 `= S.inl (cocycle` σ g_1 g_2`)` `*` σ `(`g_1 `*` g_2`)` `*` σ g_3
	Triple product decomposition via the cocycle.
`section_triple_product'`	σ g_1 `*` σ g_2 `*` σ g_3 `= S.inl (cocycle` σ g_1 g_2`)` `* S.inl (cocycle` σ `(`g_1`*`g_2`)` g_3`)` `*` σ `(`g_1`*`g_2`*`g_3`)`
	Iterated triple product decomposition.

3.1 Lean code: core definitions

The following shows the proposed Mathlib-style definitions. In our development, the corresponding definitions use different names (`sectionCocycle`, `sectionDiff`, `sectionConjAct`) at the top level of the development namespace. For the Mathlib PR, these would be renamed to use dot notation under `GroupExtension.Section` and placed in a new file `Mathlib/GroupTheory/GroupExtension/Cocycle.lean`.

```
variable {N E G : Type*} [Group N] [Group E] [Group G]
variable (S : GroupExtension N E G)

namespace GroupExtension.Section

variable {S} (σ : S.Section)

noncomputable def cocycle (g₁ g₂ : G) : N :=
  (mul_mul_mul_inv_mem_range_inl σg₁ g₂).choose

@[simp]
theorem cocycle_spec (g₁ g₂ : G) :
    S.inl (cocycle σg₁ g₂) =
      σ g₁ * σ g₂ * (σ (g₁ * g₂))⁻¹ :=
  (mul_mul_mul_inv_mem_range_inl σg₁ g₂).choose_spec

theorem mul_eq_cocycle_mul (g₁ g₂ : G) :
    σ g₁ * σ g₂ =
      S.inl (cocycle σg₁ g₂) * σ(g₁ * g₂) := by
  have h := cocycle_spec σg₁ g₂
  rw [eq_mul_inv_iff_mul_eq] at h
  exact h.symm
```

```
end GroupExtension.Section
```

Note on membership lemma reuse. The definition above uses `mul_mul_mul_inv_mem_range_inl` from `Basic.lean` directly, rather than re-deriving the membership fact. Similarly, `Section.diff` would use `mul_inv_mem_range_inl`. See Section 2.2.

3.2 Lean code: the splitting criterion

```
namespace GroupExtension

theorem splits_iff_trivial_cocycle :
    Nonempty S.Splitting ↔
    ∃ σ : S.Section, ∀g₁ g₂ : G,
      Section.cocycle σg₁ g₂ = 1 := by
  constructor
  · rintro ⟨s⟩
    refine ⟨s.toSection, fun g₁ g₂ => ?_⟩
    apply S.inl_injective
    rw [Section.cocycle_spec, map_one]
    show s g₁ * s g₂ * (s (g₁ * g₂))⁻¹ = 1
    rw [map_mul s, mul_inv_cancel]
  · rintro ⟨σ, hσ⟩
    refine ⟨{
      toFun := σ
      map_one' := by
        have spec := Section.mul_eq_cocycle_mul σ1 1
        rw [hσ 1 1, map_one, one_mul, mul_one] at spec
        have h : σ1 * σ1 = σ1 := spec
        calc σ1
            = σ 1 * σ1 * (σ 1)⁻¹ := by group
          _ = σ 1 * (σ 1)⁻¹ := by rw [h]
          _ = 1 := mul_inv_cancel _
      map_mul' := fun g₁ g₂ => by
        have spec := Section.mul_eq_cocycle_mul σg₁ g₂
        rw [hσ g₁ g₂, map_one, one_mul] at spec
        exact spec.symm
      rightInverse_rightHom := σ.rightInverse_rightHom
    }⟩

end GroupExtension
```

4 Relationship to Existing Infrastructure

4.1 Dependency on `GroupExtension.Defs`

Our contribution depends on the following from `Mathlib.GroupTheory.GroupExtension.Defs`:

- `GroupExtension N E G`: The structure representing a short exact sequence $1 \to N \to E \to G \to 1$, consisting of `inl : N →* E`, `rightHom : E →* G`, injectivity of `inl`, exactness (`range_inl_eq_ker_rightHom`), and surjectivity of `rightHom`.
- `GroupExtension.Section`: A set-theoretic right inverse of `rightHom`, i.e., a function $\sigma : G \to E$ with $\pi \circ \sigma = \mathrm{id}_G$.

- `GroupExtension.Splitting`: A *homomorphic* section—a group homomorphism $s : G \to E$ that is also a right inverse of `rightHom`.
- `GroupExtension.conjAct : E →* MulAut N`: The conjugation action of E on N through the extension. Our `Section.conjAct` composes this with a section to get $G \to$ `MulAut` N.
- `GroupExtension.inl_conjAct_comm`: The specification `S.inl(S.conjAct(e)(n))` $= e \cdot$ `S.inl(n)` $\cdot e^{-1}$. This is the key lemma used in our `Section.conjAct_spec` and in the 2-cocycle identity proof.
- `GroupExtension.IsConj`: N-conjugacy of splittings (relevant for the H^1 bridge in Phase 2).

4.2 Dependency on `GroupExtension.Basic`

From `Mathlib.GroupTheory.GroupExtension.Basic` we use:

- `GroupExtension.surjInvRightHom`: The canonical section obtained from `Function.surjInv` applied to the surjectivity of `rightHom`.
- `GroupExtension.ConjClasses`: The quotient by N-conjugacy (relevant for Phase 2).
- `Section.mul_mul_mul_inv_mem_range_inl`: The membership lemma establishing that $\sigma(g_1) \cdot \sigma(g_2) \cdot \sigma(g_1 g_2)^{-1} \in$ range(`inl`). Our cocycle definition uses this directly as the witness for `Exists.choose`.
- `Section.mul_inv_mem_range_inl`: The membership lemma establishing that $\sigma(g) \cdot \sigma'(g)^{-1} \in$ range(`inl`). Our section difference definition uses this directly.
- `Splitting.conjAct : G →* MulAut N`: The conjugation action as a group homomorphism, obtained by composing `GroupExtension.conjAct` with a splitting. Our `Section.conjAct` generalizes this to set-theoretic sections (losing the homomorphism property).

4.3 Relationship to `GroupCohomology.LowDegree`

Mathlib's `Mathlib.RepresentationTheory.Homological.GroupCohomology.LowDegree` provides low-degree specializations of group cohomology:

- `groupCohomology.H1π`: Epimorphism from 1-cocycles to H^1 (the group H^1 itself is defined in `GroupCohomology.Basic`).
- `groupCohomology.H2π`: Epimorphism from 2-cocycles to H^2.
- `IsMulCocycle`$_2$: The multiplicative 2-cocycle condition (takes `f : G × G → M` with a `MulDistribMulAction G M`).
- `IsMulCoboundary`$_2$: The multiplicative 2-coboundary condition.
- `cocyclesOfIsMulCocycle`$_2$: Converts a multiplicative 2-cocycle into an element of `cocycles`$_2$ for the `Rep.ofMulDistribMulAction`-induced representation.

Our `Section.cocycle_mul_eq` proves that the section cocycle satisfies the multiplicative 2-cocycle identity in curried form with respect to the `MulAut`-valued conjugation action. The Phase 2 bridge would show that:

1. The section cocycle, uncurried and equipped with the `MulDistribMulAction` extracted from the `MulAut`-valued action, satisfies `IsMulCocycle`$_2$.
2. Changing the section changes the cocycle by an `IsMulCoboundary`$_2$.
3. The resulting cohomology class in H^2 classifies the extension up to equivalence.

Type-theoretic gaps. Several type-level mismatches must be bridged:

1. **Curried vs. product:** Our cocycle has type `G → G → N` (curried), while Mathlib's `IsMulCocycle`$_2$ expects `G × G → M` (uncurried product type). The bridge requires `Function.uncurry`.
2. **MulAut vs. SMul:** Our conjugation action produces `MulAut N`, while `IsMulCocycle`$_2$ uses `g • f(h, j)` where • is a `MulDistribMulAction`. The bridge requires extracting the `MulDistribMulAction`

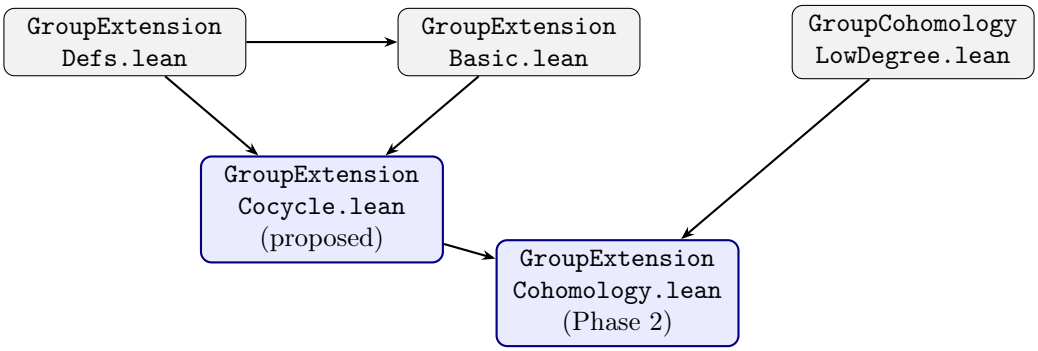
from the `MulAut`-valued homomorphism.

3. **Multiplicative vs. additive:** Mathlib's `H2` is defined via `Rep k G` (k-linear G-representations). For abelian N, the bridge requires coercing between `Multiplicative`/`Additive` and constructing the appropriate representation via `Rep.ofMulDistribMulAction`. The existing `cocyclesOfIsMulCocycle₂` in `LowDegree.lean` handles part of this.

These coercions are the main technical challenge for Phase 2.

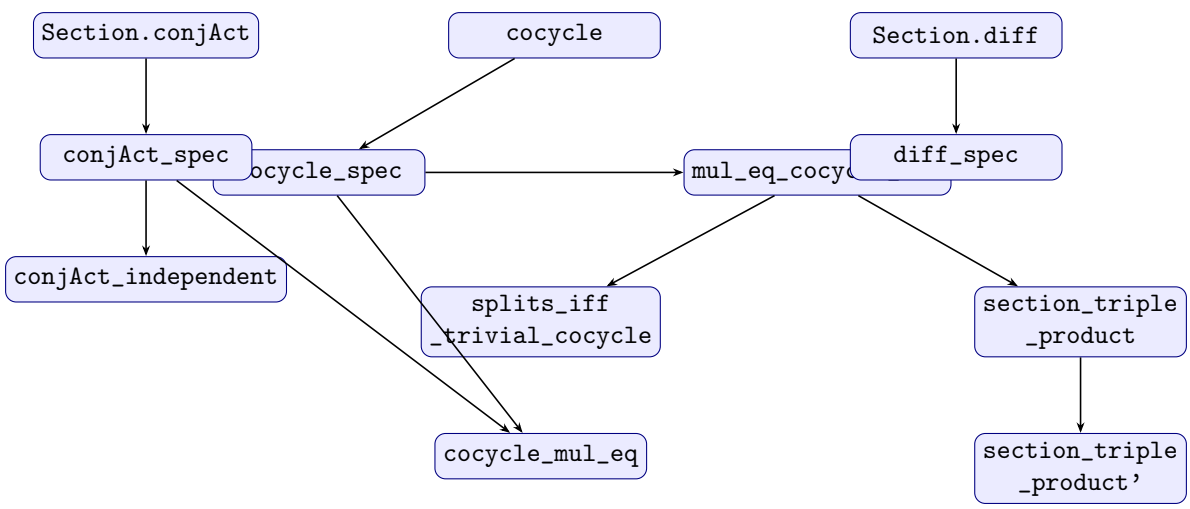
4.4 Composition diagram

The following diagram shows how our additions compose with existing Mathlib infrastructure.



5 Dependency Structure and Proof Architecture

The internal dependency structure of our proposed theorems is shown below. An arrow $A \to B$ means B depends on A in its proof.



5.1 Proof strategy overview

The proofs follow a uniform strategy:

1. **Membership lemma**: Reuse the existing Mathlib lemma (`mul_mul_mul_inv_mem_range_inl` or `mul_inv_mem_range_inl`) establishing that the relevant expression lies in range(`inl`) = ker(`rightHom`).

2. **Definition via `choose`**: Extract the N-valued preimage using `Exists.choose` on the membership proof.
3. **Specification via `choose_spec`**: The specification lemma (`cocycle_spec`, `diff_spec`) is simply `Exists.choose_spec`.
4. **Downstream theorems via injectivity**: All equalities between N-valued expressions are proved by applying `S.inl_injective` and then rewriting using the specification lemmas. This reduces N-level reasoning to E-level group algebra.

This "push through `inl`, reason in E, pull back via injectivity" pattern is the key proof engineering technique. It avoids working directly with the noncomputable `choose` and instead works entirely through the specification interface.

6 Toward H^1 and H^2: Phase 2 Roadmap

6.1 Phase 2a: The H^1 bridge

Target theorem:

$$\texttt{GroupExtension.ConjClasses} \simeq H^1(G, N) \qquad \text{(for abelian } N\text{, split extension)}$$

What we provide (Phase 1):

- `Section.conjAct`: The G-action on N via sections.
- `Section.conjAct_independent`: For abelian N, the action is section-independent (so it descends to a well-defined G-module structure).
- `Section.diff`: The difference function between sections.

What remains (Phase 2):

- Show that the difference function between two splittings is a 1-cocycle.
- Show that N-conjugation changes the splitting by a 1-coboundary.
- Construct the bijection between `ConjClasses` and H^1.

6.2 Phase 2b: The H^2 bridge

Target theorem:

$$\{\text{equivalence classes of extensions with kernel } N \text{ and action } \varphi\} \simeq H^2(G, N)$$

What we provide (Phase 1):

- `Section.cocycle`: The N-valued 2-cocycle.
- `Section.cocycle_mul_eq`: The 2-cocycle identity.
- `Section.diff`: The section difference function.
- `splits_iff_trivial_cocycle`: Splitting iff trivial cocycle class.

What remains (Phase 2):

- **Cocycle change formula**: For sections σ, σ' with difference $d = \mathrm{diff}(\sigma, \sigma')$, prove

$$c_{\sigma'}(g_1, g_2) = d(g_1) \cdot \varphi(g_1)(d(g_2)) \cdot c_\sigma(g_1, g_2) \cdot d(g_1 g_2)^{-1}$$

  showing the cocycles differ by a coboundary.
- **Type-theoretic bridge**: For abelian N, coerce the multiplicative cocycle into the setting required by Mathlib's `Rep k G`-based `H2`. This involves: (i) uncurrying via `Function.uncurry`, (ii) extracting a `MulDistribMulAction` from the `MulAut`-valued conjugation action, and (iii) applying $\texttt{cocyclesOfIsMulCocycle}_2$ to obtain an element of $\texttt{cocycles}_2$.
- **Extension equivalence**: Define equivalence of extensions (with fixed kernel and quotient) and construct the bijection with H^2.

6.3 Estimated effort

Phase	Scope	Status
Phase 1 (this PR)	Cocycle + splitting criterion	Complete (zero sorry)
Phase 2a (H^1)	Conjugacy classes $\leftrightarrow H^1$	Future work
Phase 2b (H^2)	Extension classes $\leftrightarrow H^2$	Future work

The main bottleneck for Phase 2b is the type-theoretic bridge between multiplicative N-valued cocycles and Mathlib's k-linear representation-theoretic H^2. If this bridge proves too heavy, a fallback is to define a multiplicative H^2_{mul} directly (as the quotient of multiplicative 2-cocycles by multiplicative 2-coboundaries) and prove the classification theorem in that setting, deferring the connection to Mathlib's `groupCohomology.H2` to a subsequent PR.

7 Testing and Compatibility

7.1 Build status

The development builds successfully against Mathlib v4.29.0-rc6 (Lean 4 toolchain `leanprover/lean4:v4.29.0-rc6`). All files compile with `lake build` with no errors and zero `sorry`.

7.2 Proof status

All theorems in the development are **fully verified** by the Lean type checker with **zero** `sorry`. In particular, the 2-cocycle identity (`Section.cocycle_mul_eq`) is completely proved.

The proof proceeds by applying `S.inl_injective` to reduce the N-level equality to an E-level equality, then uses a pair of auxiliary `have` lemmas (`lhs` and `rhs`) that each simplify one side via `section_mul_eq` rewrites and associativity/cancellation steps. The final step applies `mul_right_cancel` to equate the two sides.

```
theorem sectionCocycle_isMulCocycle₂_conj
    (σ : S.Section) (g₁ g₂ g₃ : G) :
    sectionCocycle S σg₁ g₂ *
      sectionCocycle S σ(g₁ * g₂) g₃ =
    sectionConjAct S σg₁
      (sectionCocycle S σg₂ g₃) *
      sectionCocycle S σg₁ (g₂ * g₃) := by
  apply S.inl_injective
  rw [map_mul, map_mul, inl_sectionConjAct]
  have lhs : ... := by
    rw [mul_assoc, ←section_mul_eq S σ(g₁ * g₂) g₃,
        ←mul_assoc, ←section_mul_eq S σg₁ g₂]
  have rhs : ... := by
    rw [mul_assoc _ (S.inl ...)]
    rw [← section_mul_eq S σg₁ (g₂ * g₃)]
    rw [mul_assoc _ (σ.toFun g₁)⁻¹, inv_mul_cancel_left]
    rw [mul_assoc, ←section_mul_eq S σg₂ g₃,
        ←mul_assoc]
  apply mul_right_cancel
    (b := σ.toFun (g₁ * g₂ * g₃))
  rw [lhs, show g₁ * g₂ * g₃ = g₁ * (g₂ * g₃)
        from mul_assoc g₁ g₂ g₃, rhs]
```

Note on factor order. The correct statement places $c(g_1, g_2)$ before $c(g_1 g_2, g_3)$ on the left-hand side. An earlier draft had the factors reversed; correcting the order was the key step that enabled the proof to close.

7.3 Module structure

The proposed Mathlib file `Mathlib/GroupTheory/GroupExtension/Cocycle.lean` would contain all definitions and theorems from Tables 1 and 2. The file structure follows Mathlib conventions:

```
/-
  GroupExtension.Cocycle -- Section cocycles and
  splitting criteria for group extensions.

  ## Main definitions

  * ‘GroupExtension.Section.cocycle‘
  * ‘GroupExtension.Section.diff‘
  * ‘GroupExtension.Section.conjAct‘

  ## Main results

  * ‘GroupExtension.splits_iff_trivial_cocycle‘
  * ‘GroupExtension.Section.cocycle_spec‘
  * ‘GroupExtension.Section.cocycle_mul_eq‘

  ## References

  See the TODO in ‘GroupExtension/Defs.lean‘ (lines 43-52).
  This module is Phase 1 toward completing the H1/H2
  classification theorems documented there.
-/

import Mathlib.GroupTheory.GroupExtension.Basic
```

7.4 Interaction with existing `simp` lemmas

We have verified that our proposed `@[simp]` lemmas do not conflict with existing Mathlib `simp` lemmas for `GroupExtension`. The key observation is that our `simp` lemmas rewrite `S.inl (cocycle ...)` and `S.inl (diff ...)`, which are new term patterns not matched by any existing `simp` lemma. There is no risk of `simp` loops.

7.5 AI use disclosure

Per Mathlib's policy on AI-assisted contributions, the PR description will disclose that AI tools (Claude) were used in the development of this contribution, including proof exploration, tactic suggestion, and documentation drafting. All mathematical content has been verified by the Lean type checker with zero `sorry`.

References

[1] K. S. Brown, *Cohomology of Groups*, Graduate Texts in Mathematics, vol. 87, Springer-Verlag, 1982. https://doi.org/10.1007/978-1-4684-9327-6

[2] Mathlib Contributors, *Mathlib.GroupTheory.GroupExtension.Defs*, 2024–2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/GroupTheory/GroupExtension/Defs.html

[3] Mathlib Contributors, *Mathlib.GroupTheory.GroupExtension.Basic*, 2024–2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/GroupTheory/GroupExtension/Basic.html

[4] Mathlib Contributors, *Mathlib.RepresentationTheory.Homological.GroupCohomology.LowDegree*, 2024–2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/RepresentationTheory/Homological/GroupCohomology/LowDegree.html

[5] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Automated Deduction – CADE 28*, LNCS, vol. 12699, Springer, 2021, pp. 625–635.